From Kneser-Ney to Causal Self-Attention: Language Models, Embeddings and Representations on WikiText-2

Nabil Rakaiza Abror (A0305715J)

Code: `https://github.com/nabilrakaiza/papertrail`

1 Setup

This report asks how four families of language model (an interpolated Kneser-Ney trigram, an RNN, an LSTM and a Transformer) compare at next-token prediction on WikiText-2 (Part I); whether fixed pretrained input embeddings help or hurt compared with embeddings learned jointly with the model (Part II); and whether a better language model also gives better representations for a downstream task, topic classification on AG News (Part III).

Data. WikiText-2 [7] (`wikitext-2-v1`, the pre-tokenized release, via Hugging Face `datasets` [4]) with its official splits, lowercased, empty lines dropped and every line ended with `<eos>`. The vocabulary comes from the training split only (28,913 types including `<pad>`, `<unk>` and `<eos>`); other words become `<unk>`. Each split is one token stream (2.08M training, 216K validation and 244K test tokens), read by the neural models in contiguous 64-token chunks.

Evaluation. Every model is scored on the same 244,101 test targets (every test token except the first) by mean next-token cross-entropy; perplexity is its exponential. Padding, which appears only in the final chunk, is excluded. At evaluation the RNN and LSTM start every chunk from a zero state, so all neural models see at most 64 tokens of history; the trigram sees two. All results are from one seed (42) on an Apple M1 laptop (8 GB): the neural models on the `mps` device with PyTorch 2.9 [10], and the trigram, skip-gram training and probe classifiers on the CPU.

2 Part I: Model families

2.1 Models

Interpolated Kneser-Ney trigram. Interpolation mixes trigram, bigram and unigram estimates, and the lower orders matter most when the trigram has no evidence. With ordinary interpolation they are based on raw counts, which misleads for a word that is frequent only after one particular word: "Francisco" is common, but almost always after "San", so it is a poor guess in a new context. Kneser-Ney smoothing [3, 1] instead bases the lower orders on continuation counts, the number of distinct words a word has been seen after, a better measure of how likely it is in an unseen context. We use one fixed absolute discount $D = 0.75$ at the trigram and bigram levels; the unigram level is the undiscounted continuation distribution.

The order is three. A 4-gram model would not help on a corpus of this size because of data sparsity: on the training text, 85% of the 1.38M distinct trigrams already occur exactly once, and for 4-grams, 1.81M of the 2.08M occurrences are distinct and 94% of those occur once (`results/analysis_part1.json`). Most 4-gram contexts met at test time would therefore be unseen or rare, so the model would mostly fall back to its trigram estimate anyway.

Neural models. A 2-layer Elman RNN [8] (tanh) and a 2-layer LSTM [2], both with hidden size 512, dropout 0.3 and PyTorch's default initialization, and a decoder-only Transformer [15] with 4 blocks, width 200, 4 heads, feed-forward size 800, dropout 0.2, post-LN, sinusoidal positions and weights initialized from a normal distribution with standard deviation 0.02. Each has randomly initialized, trainable 200-dimensional input embeddings (the size of the GloVe vectors in Part II) and an untied output layer, its largest part (14.8M parameters in the RNN and LSTM, 5.8M in the Transformer). All use AdamW [6] (weight decay 0.01), batch size 32, 64-token chunks read in corpus order so the RNN and LSTM carry their state across batches (truncated backpropagation through time), gradient clipping at 1.0, and at most 20 epochs with early stopping after 3 without improvement, keeping the best validation checkpoint. The learning rate is a constant 10^{-3} for the

RNN and LSTM, and 5×10^{-4} with 1,000 warm-up steps and cosine decay over the 20-epoch budget for the Transformer, since a post-LN Transformer tends to diverge without warm-up. Every model predicts each target from preceding tokens only; for each neural model we checked that changing one token leaves every earlier prediction unchanged.

Fairness. The models come from different families (counting, recurrence and attention), so their sizes are not matched: each was sized to train in reasonable time on the laptop, and none was tuned. The training setup around them is shared (optimizer, epoch budget and early stopping, context length and evaluation), but the Transformer differs in its learning-rate schedule, dropout and initialization.

2.2 Results

Model	Params	Val PPL	Test PPL	Epochs (best)	s / epoch	Time to best	Inference (tok/s)
KN trigram	1.38M trigrams	240.66	226.00	–	–	10 s	453,082 / 9*
RNN	21.5M	268.92	245.14	16 (13)	475	103 min	21,084
LSTM	24.2M	226.13	210.84	10 (7)	480	56 min	19,405
Transformer	13.5M	160.41	**149.52**	9 (6)	360	36 min	**32,718**

Table 1: Part I results (`results/*.json`, `results/benchmark.json`). Time per epoch is the median training step times 1,013 steps, measured for all models under the same conditions; time to best is best epoch × time per epoch, and for the trigram the time to count n-grams on the CPU. Inference: test targets per second for a full forward pass at batch size 32, after 10 warm-up batches, mean of 3 repeats. *The trigram can score a known target with a few lookups (453,082 tok/s) but producing a full distribution, as the neural models always do, runs at 9 positions per second in our pure-Python implementation.

Learning behaviour. In all three neural models training loss keeps falling while validation loss levels off (Figure 1; training loss is measured with dropout on, so the true gap is larger). All three overfit: by the training–validation gap at the best epoch the Transformer most (0.78 nats, against 0.67 for the LSTM and 0.49 for the RNN), and by how far validation loss rises after it the LSTM (+0.088, against +0.024 for the Transformer and +0.010 for the RNN); early stopping keeps the checkpoint before the rise. The Transformer has the lowest training loss at every epoch from the second on, yet still the lowest validation loss. One possible explanation is that attention gives it direct access to every earlier token in the window, while a recurrent model must squeeze its history into one fixed-size state, so it can fit the training text more closely; its lower dropout could also contribute. The trigram has no curve but also memorizes: its perplexity on its own training text is 16.9, against 226.00 on test, and its discount plays the role of dropout and early stopping.

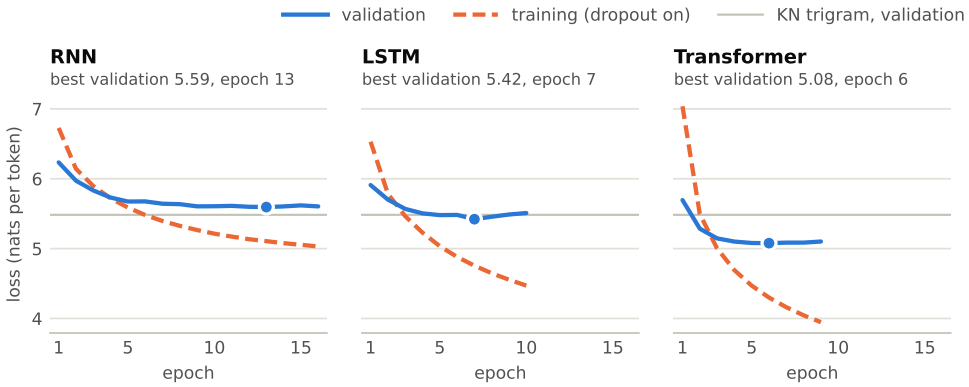


Figure 1: Training and validation loss per epoch. The dot marks the kept checkpoint; the grey line is the trigram's validation loss.

Generated text. All models continue the same five prompts (temperature 0.8, top-k 40, 50 new tokens; one sampling function, re-seeded for every continuation; samples in `results/generations.json` and Appendix A.1). The trigram's samples are locally fluent over two or three words but have no structure beyond that and drift off topic. The RNN's are more fluent locally but lose the thread within a sentence or two; the LSTM's hold together slightly better but still drift, most clearly for "the species is found in"; the Transformer's are the most fluent, locally and across sentences. On repetition, two trigram prompts ending in "in" led to identical continuations (once both had sampled "the", its two-token context and the random draws coincided); among the neural models the Transformer repeats most (share of distinct bigrams 0.90, against 0.93 to 0.94;

"766th" three times in one sample). The trigram produces the most `<unk>` (6.2 per sample, against 3.0 to 4.0); with five samples per model, small differences mean little.

2.3 How much history each model uses

Splitting the test loss by how many tokens of history each prediction had, that is, by position within the 64-token chunk (Figure 3 in Appendix A.2; `results/analysis_part1.json`): With one token of history the RNN and LSTM are far worse than the Transformer and the trigram (perplexity 1,637 and 1,624, against 366 and 227). With more history all three neural models keep improving (from positions 8–15 to 32–63 their loss falls by 0.108, 0.181 and 0.132 nats) while the trigram stays flat; averaged over positions 32–63, the RNN is level with the trigram (224 against 225), the LSTM is at 184 and the Transformer at 138.

Two further protocols (Appendix A.2) agree: when no prediction is scored with fewer than 33 tokens of history, the RNN moves marginally ahead of the trigram (224.56 against 226.00) and the other rankings are unchanged; when the recurrent models carry their state across the whole test set, the LSTM improves by a further 18.5 perplexity points and the RNN by 3.2.

2.4 Tradeoffs

Observed. The Transformer has the lowest test perplexity (149.52), followed by the LSTM (210.84), the trigram (226.00) and the RNN (245.14). The trigram is far cheaper than any neural model to fit (seconds) and to score known text. The LSTM has slightly more parameters than the RNN (24.2M against 21.5M) and costs the same per epoch, but reaches a lower perplexity, reaches its best checkpoint in about half the time because it needs fewer epochs, and gains much more from longer history (Table 4). The Transformer has the fewest parameters, is the fastest per epoch, to its best checkpoint and at inference, and has the lowest perplexity: among the neural models on this setup there is no tradeoff. This holds for what was tested: WikiText-2, models of the sizes in Section 2.1, a laptop's training budget and a single seed.

Possible explanations. The LSTM's advantage over the RNN is consistent with its gating, which lets gradients and information pass across many steps without vanishing. The history results fit this: within the window the RNN gains least from extra history, and with its state carried across the test set it improves by only 3.2 points against the LSTM's 18.5, a measure of how much each state retains over long spans (both were trained with backpropagation through only 64 tokens, so this reflects what each learned to retain, not gradient flow beyond 64 steps). The RNN's loss to the trigram is best explained by a cold start: in training the RNN and LSTM carried their state across batches and so almost never predicted from a zero state, which is how every evaluation chunk begins, while the stateless Transformer was trained and evaluated on chunks alike. Lack of history alone does not explain it: with one token of history the Transformer (366) and the trigram (227) do far better. The Transformer's lead is consistent with attention seeing every earlier token in the window directly, rather than through a fixed-size recurrent state. Part of its lead comes from the chunk start, but not all of it (138.79 against 184.32 and 224.56 with that removed). Its parameter count (the smallest) and context length (the same) do not explain it; its learning-rate schedule, lower dropout and initialization cannot be ruled out.

3 Part II: Embedding variants

3.1 Setup

The Part I LSTM is trained with three input-embedding settings; everything else (architecture, embedding size, data, training procedure and seed, and so every other initial weight) is identical. *(1) Random, trainable:* the Part I LSTM itself. *(2) Own skip-gram, frozen:* 200-dimensional skip-gram vectors [9] trained with gensim [13] (window 5, 10 negative samples, 10 epochs) on the WikiText-2 training split only, covering every token except `<pad>`; training them took 233 s on the CPU, on top of the language-model training. *(3) GloVe, frozen:* the public `glove.6B.200d` vectors [12] (6B tokens of Wikipedia and Gigaword), covering 96.2% of the vocabulary and 97.4% of the training text; the most frequent missing tokens are `<eos>` and the markers `@-@`, `@.@` and `@,@`. Missing tokens get seeded random rows with the found vectors' standard deviation, and the *entire* table is frozen (checked bit-for-bit unchanged after training), leaving 18.4M trainable parameters instead of 24.2M.

Settings 1 and 2 learn from the same text and differ in whether the embeddings are learned jointly and kept trainable, or learned separately and frozen. Settings 2 and 3 are both learned separately and frozen and differ in the data (2M WikiText tokens against 6B tokens of general text), but also in the method, so they do not isolate the data alone.

Input embeddings	Trainable params	Coverage	Val PPL	Test PPL	Epochs (best)	s / epoch
Random, trainable	24.2M	–	226.13	210.84	10 (7)	480
Own skip-gram, frozen	18.4M	100%	155.29	**147.01**	13 (10)	452
GloVe, frozen	18.4M	96.2%	178.42	167.55	13 (10)	433

Table 2: Part II results (`results/lstm*.json`). Coverage is the share of vocabulary tokens with a pretrained vector. Time per epoch is measured as in Table 1.

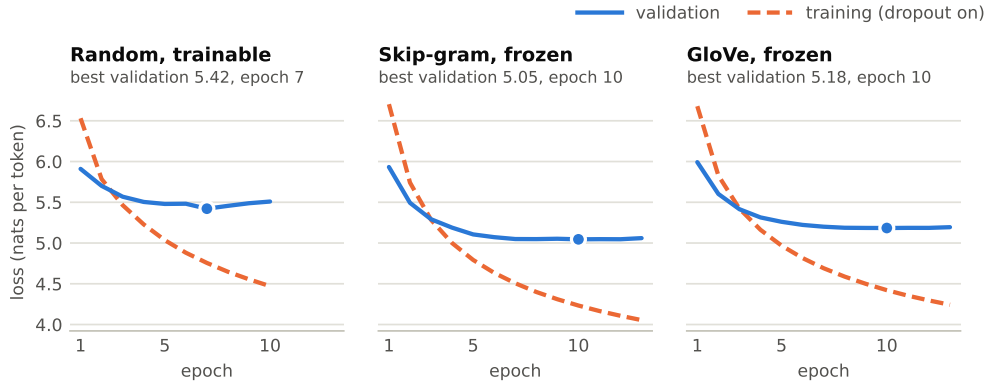


Figure 2: Training and validation loss per epoch for the three embedding settings. The dot marks the kept checkpoint.

3.2 Results

Prediction quality. Both frozen settings beat the random, trainable one by a wide margin: test perplexity 147.01 with the skip-gram vectors and 167.55 with GloVe, against 210.84. The frozen skip-gram LSTM is also slightly better than the Part I Transformer (149.52).

Convergence and stability. After one epoch the three settings are level (validation loss 5.91, 5.93 and 5.99). From the second epoch on both frozen settings are lower at every epoch and keep improving for longer: their best epoch is 10, against 7 for the random setting, whose validation loss then rises (Figure 2). All curves are smooth after the first epoch; the random setting's best epoch is a small isolated dip. Time per epoch is similar (452 and 433 s against 480 s, within the variation between timing rounds).

Generated text. With the same prompts and decoding as in Part I (`results/generations_part2.json`), the random setting's samples tend to drift off topic, while both frozen settings stay on topic more consistently ("the species is found in" continues with genus and habitats under skip-gram, and with birds under GloVe) and are more fluent locally. They also repeat more within a sample (share of distinct bigrams 0.92 and 0.91, against 0.94; the skip-gram species sample uses "species" four times), and GloVe produces the most `<unk>` (5.2 per sample, against 2.0 and 3.0).

3.3 Interpretation

Trainability (settings 1 and 2). Learning the embeddings separately and freezing them helps here. One possible reason is stability: with a frozen table the LSTM's inputs stop moving, while in the random setting the table and the model must be learned at once from the same data. The frozen settings also overfit later: their best epoch comes three epochs later, at a lower validation loss, and validation loss rises much less after it (+0.015 and +0.010 nats, against +0.088), although their training–validation gap at the best epoch is larger (0.81 and 0.76, against 0.67). A second, simpler reason may contribute: freezing leaves 5.8M fewer parameters to fit from the same 2.08M tokens, so less room to memorize. The two are compatible, and this experiment cannot separate them.

Extra data (settings 2 and 3). The skip-gram vectors were trained on the very corpus the language model is trained and tested on, so they match its vocabulary, tokenization and topics; GloVe was trained on different, differently tokenized text. For the LSTM that match appears to outweigh GloVe's roughly 3,000 times more data (the two also differ in method; on the Transformer the order reverses, but that comparison is confounded, below). It shows in coverage too: GloVe lacks `<eos>` and the `@-@`, `@.@` and `@,@` markers, among the most frequent tokens here, which keep random, frozen rows. GloVe's extra data may also overlap the test set: its 2014 Wikipedia text likely includes many of the articles WikiText-2 was built from, so it may have seen some validation and test text indirectly; for the LSTM it still loses to skip-gram, so this did not decide that comparison. On text closer to GloVe's own we would expect GloVe to do better; that was not tested.

The same embeddings on the Transformer. As an optional extension (Appendix A.3), the Transformer was trained with the same tables frozen in, rescaled to its own initial scale. It ends at test perplexity 131.17 (skip-gram) and 127.45 (GloVe) against 149.52, but not like-for-like: its learning rate decays over the 20-epoch budget, the Part I Transformer early-stopped at epoch 9 (its best checkpoint still at 84% of the peak rate), and both frozen runs trained all 20 epochs, into the low-rate end. At epoch 6 their validation perplexity is 156.78 and 160.36, against the baseline's 160.41, so most of the gain comes from freezing letting the model train longer without overfitting, and GloVe overtakes skip-gram only late; which vectors suit the Transformer better is not settled here. If the reversal is real, one possible reason is that the difference between embedding methods reflects their hyperparameters (dimension, window, training data) more than the algorithms [5], so the winner need not be stable across models.

A confound: scale. The three tables also start at different scales: the random table has standard deviation 1.0 (PyTorch's default), the skip-gram vectors 0.19 and GloVe 0.43. The initial scale matters in itself, so part of the gap between the random setting and the frozen ones may come from scale rather than from what the vectors contain. This experiment cannot separate the two; a random, trainable table started at the pretrained vectors' scale would.

4 Part III: Downstream representations

4.1 Setup

Task and data. Topic classification on AG News [16] (World, Sports, Business, Sci/Tech; balanced, so chance is 25%): a class-balanced, seeded sample of 10,000 training items and a disjoint 2,000 for validation from the official training split, and the official test split (7,600), used once. Items are lowercased, tokenized following WikiText-2's conventions, mapped to its vocabulary (7.0% of test tokens become `<unk>`) and cut to the 64-token training length (3.8% of test items are longer).

Representations and classifier. Six frozen language models are probed: the Part I LSTM and Transformer and the four frozen-embedding variants (Part II). Each reads every item from a fresh state; its final-layer hidden states (512 numbers per token for the LSTMs, 200 for the Transformers) become one vector per item by *mean pooling* over the real tokens, excluding padding, or by *last-token pooling*, the state at the last real token, which has read the whole item (padding comes after the item, so it cannot affect these states). The classifier is the same for all twelve: multinomial logistic regression (scikit-learn [11]) on standardized features (fitted on training items only), with the regularization strength chosen by validation macro-F1 from $\{0.01, 0.1, 1, 10\}$.

4.2 Results

		Mean pooling		Last-token pooling	
Language model	LM test PPL	Accuracy	Macro-F1	Accuracy	Macro-F1
LSTM (Part I)	210.84	62.89%	0.628	47.62%	0.475
Transformer (Part I)	149.52	78.45%	0.784	72.83%	0.727
LSTM, skip-gram frozen	147.01	79.39%	0.794	74.54%	0.745
LSTM, GloVe frozen	167.55	83.18%	0.831	77.80%	0.778
Transformer, skip-gram frozen	131.17	81.92%	0.819	80.13%	0.801
Transformer, GloVe frozen	127.45	**85.64%**	**0.856**	**84.24%**	**0.842**

Table 3: Part III results on the AG News test set (`results/probe_agnews.json` for the two Part I models, `results/probe_agnews_extended.json` for all six; the two files agree on the Part I models).

Which representation works better. Mean pooling beats last-token pooling for all six models, most for the Part I LSTM (15.3 points) and least for the frozen-embedding Transformers (1.4 to 1.8). With the embeddings held fixed the Transformer beats the LSTM every time, under both poolings (although the LSTMs give 512 features against 200, and the frozen Transformers trained longer), and frozen pretrained embeddings raise accuracy sharply: the LSTM goes from 62.89% to 79.39% with skip-gram and 83.18% with GloVe, and the best is the Transformer with GloVe (85.64%). For 84% of test items the last real token is a full stop, so last-token pooling takes the state just after a sentence ends, whose job is mostly to predict what follows rather than to describe the item; the topic is spread over the item, and averaging keeps it. The pooling gap also fits Part I's architectural difference: in the LSTM, early information must pass through every later step to reach the final state, and each state compresses all earlier history into one fixed-size vector, while each Transformer position attends to the earlier tokens directly.

Errors. Across the four probes of the Part I models, Sports is easiest and Business and Sci/Tech hardest (best probe: F1 0.89 for Sports, 0.80 World, 0.73 Sci/Tech, 0.71 Business), and the largest confusions are between Business and Sci/Tech (331 and 320 items). The most confident errors (Appendix A.4) include sports stories labelled World, arguably labelling problems; items that fit both Business and Sci/Tech; and a few plain model errors. Unknown words give the Business and Sci/Tech confusion only weak support (Appendix A.4); ambiguity is more likely: much news, such as a technology company's earnings, is as much Business as Sci/Tech, and with a single hard label per item rather than a distribution over classes, such items will be confused whichever representation is used.

Language-model ranking against downstream ranking. Between the two Part I models the rankings agree. Across all six they mostly agree, but not fully: the Spearman rank correlation between test perplexity and test accuracy is 0.66 with mean pooling and 0.83 with last-token pooling. The clearest exception is the LSTM with frozen GloVe, fifth of six in perplexity (167.55) but second in classification with mean pooling (83.18%); GloVe gives the better features for both architectures. So better next-token prediction goes with better features in general here, but the pretrained vectors a model starts from can matter more downstream than its perplexity suggests; six models are a small sample. A possible reason, offered as a hypothesis: GloVe's training text includes Gigaword newswire, and AG News is news, so its vectors may suit this task better; Gigaword also covers the years AG News was collected in, so some of the same stories may appear in both, which would favour GloVe as well.

5 Conclusion

On WikiText-2 at laptop scale, the Transformer is the best language model on quality and cost; the LSTM beats the RNN, which falls behind the trigram only because of how each evaluation chunk starts (Part I). Frozen pretrained embeddings help the LSTM substantially, most of all skip-gram vectors trained on the same corpus, with scale and possible test overlap as caveats; for the Transformer, freezing helps mainly by letting it train longer (Part II). Better language models mostly give better features for topic classification, but the pretrained vectors a model starts from can matter more than its perplexity (Part III). Across Parts I and III one architectural difference recurs: attention's direct access to every earlier token, against a recurrent state that must carry the whole history.

6 Limitations and lessons

Limitations. The main limitation is compute: everything ran on one laptop (Apple M1, 8 GB, no CUDA GPU), so the models are small and untuned, each run had a limited budget, and larger models are untested. *One seed:* no error bars, so small differences may be run-to-run variation. *Protocol:* resetting the recurrent state every 64 tokens caps the RNN's and LSTM's history and penalizes each chunk's start, so the headline numbers understate them. *Confounds:* in Part II the tables differ in scale as well as content, and skip-gram against GloVe changes the method as well as the data; Part III's tokenizer only approximates WikiText-2's and loses 7% of AG News tokens to `<unk>`, and six models are few for a correlation. *Transformer extension:* both runs hit the 20-epoch limit while still improving, rely on rescaled vectors, and are confounded with the learning-rate schedule. *Possible overlap:* GloVe's training text likely overlaps WikiText-2's test articles and possibly AG News stories. *Timing:* this laptop's speed varies with its power state and load, so the training times recorded during the runs are not comparable with each other; all costs here were measured again like-for-like.

Failure cases. Besides the RNN's loss to the trigram (Part I) and the doubtful labels behind many confident errors (Part III), the first LSTM run was invalid: in PyTorch 2.9 on `mps`, the built-in two-layer LSTM's inter-layer dropout keeps the mode of its first call and a fixed mask. It was caught only because a reloaded checkpoint did not reproduce its recorded loss.

Tool use. An AI coding assistant was used substantially for the code and for drafting this report from the author's notes; Appendix B states exactly how.

Lessons learned. (1) How Kneser-Ney smoothing works, and why continuation counts beat raw frequency for the lower orders. (2) A neural model is not automatically better than a well-smoothed n-gram. (3) The evaluation protocol can decide a ranking, as the cold start shows. (4) Results need checking, not just running: reloading each checkpoint exposed the dropout bug.

References

[1] S. F. Chen and J. Goodman. An empirical study of smoothing techniques for language modeling. *Computer Speech & Language*, 13(4), 1999.

[2] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8), 1997.

[3] R. Kneser and H. Ney. Improved backing-off for M-gram language modeling. *ICASSP*, 1995.

[4] Q. Lhoest et al. Datasets: A community library for natural language processing. *EMNLP System Demonstrations*, 2021.

[5] O. Levy, Y. Goldberg and I. Dagan. Improving distributional similarity with lessons learned from word embeddings. *TACL*, 3, 2015.

[6] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. *ICLR*, 2019.

[7] S. Merity, C. Xiong, J. Bradbury and R. Socher. Pointer sentinel mixture models. arXiv:1609.07843, 2016.

[8] T. Mikolov, M. Karafiát, L. Burget, J. Černocký and S. Khudanpur. Recurrent neural network based language model. *Interspeech*, 2010.

[9] T. Mikolov, K. Chen, G. Corrado and J. Dean. Efficient estimation of word representations in vector space. arXiv:1301.3781, 2013.

[10] A. Paszke et al. PyTorch: An imperative style, high-performance deep learning library. *NeurIPS*, 2019.

[11] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *JMLR*, 12, 2011.

[12] J. Pennington, R. Socher and C. D. Manning. GloVe: Global vectors for word representation. *EMNLP*, 2014.

[13] R. Řehůřek and P. Sojka. Software framework for topic modelling with large corpora. *LREC Workshop on New Challenges for NLP Frameworks*, 2010.

[14] A. Rush. The Annotated Transformer. Harvard NLP, 2018. `https://nlp.seas.harvard.edu/annotated-transformer/`

[15] A. Vaswani et al. Attention is all you need. *NeurIPS*, 2017.

[16] X. Zhang, J. Zhao and Y. LeCun. Character-level convolutional networks for text classification. *NeurIPS*, 2015.

Other code used. The Transformer is ported from the author's earlier implementation, `https://github.com/nabilrakaiza/transformer`, which was adapted from The Annotated Transformer [14].

A Additional material

A.1 Generated samples

The five prompts, the same for every model: *the album was released in*; *during the war , the*; *the species is found in*; *in the second half of the game*; *he was born in*. Part I examples, cut to their first 30 tokens (all samples: `results/generations.json`):

Prompt *the album was released in*

KN trigram the last game and the british empire casualties , " " and " the song . the ship 's cargo ship . the two `<unk>` , `<unk>` `<unk>` , and `<unk>` (a) …

RNN the 2000s season at the start of 14 august 1935 . a month of 3 : 10 , 2013 , also criticized a single film award for the game . the film …

LSTM the summer for three weeks of the uk and awards . in the episode in 2005 , the game was selected as the `<unk>` `<unk>` of the game 's best …

Transformer the united states . `<eos>` in the album , in the united states on august 25 , 2015 , the album was released as a single album . the album received …

Prompt *during the war , the*

KN trigram `<unk>` , with a `<unk>` " . however , due to an end . he has been a long time , the first half of the country , and the city , …

RNN 766th was a " `<unk>` of the length , where no can be found for each of the first time . `<eos>` the portuguese facade of the premaxillary …

LSTM `<unk>` `<unk>` were able to see the series of privacy that has been found at one of the same year . `<eos>` the episode was directed by …

Transformer 766th was in order to create a heavy emplacement at about 20 km / h (11 mph) . the 766th regiment 's regiment was made up to 40 km …

Part II, the three LSTM embedding settings (all samples: `results/generations_part2.json`):

Prompt *the species is found in*

Random, trainable the " `<unk>` " . she is located at home with his `<unk>` , which are also considered a part of the first episode of the 20th century . `<eos>` …

Skip-gram, frozen the late 2000s , and later the entire of two species have been collected by other species . the genus is derived to `<unk>` , and the genus is a …

GloVe, frozen the region that had been discovered . when a second few hours , it will be the only of the closest bird , although the kakapo was a `<unk>` . …

A.2 History and other evaluation protocols (Part I)

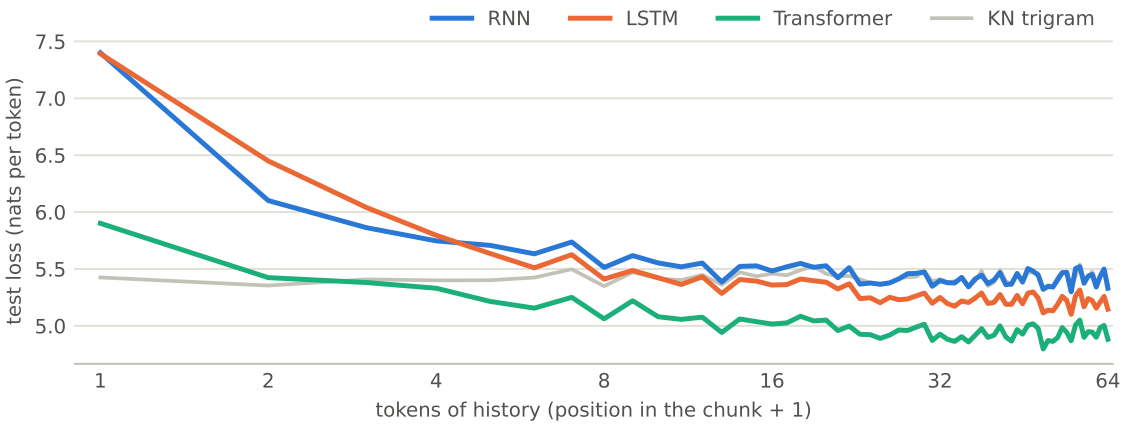


Figure 3: Test loss by the number of tokens of history, for every position of a 64-token chunk (3,815 chunks). The trigram always has two tokens of history; its line is flat.

Model	Standard (64-token chunks)	Strided (33–64 tokens of history)	Carried (unlimited history)
KN trigram	226.00	226.00	226.00
RNN	245.14	224.56	221.35
LSTM	210.84	184.32	165.82
Transformer	149.52	138.79	–

Table 4: Test perplexity of the same checkpoints under three evaluation protocols. The trigram always conditions on two tokens, so its score does not change.

Other ways of giving the models history. Table 4 scores the same test targets under two further protocols (`results/analysis_part1.json`). *Strided*: windows of 64 tokens start every 32 tokens, each from a fresh state, and only their second half is scored, so every scored prediction has between 33 and 64 tokens of history and no model's slow start is counted. *Carried*: the RNN and LSTM keep their state across the whole test stream, as in training, so their history is unlimited; the Transformer cannot do this, because it only ever attends to a fixed window.

With the slow start removed (strided), the RNN moves from behind the trigram to marginally ahead of it (224.56 against 226.00), and the ranking of the other models is unchanged. Given unlimited history (carried), the LSTM improves by a further 18.5 perplexity points, the RNN by 3.2. The Transformer keeps improving up to the end of its 64-token window; how it would behave beyond that was not measured. The standard protocol, fixed in the README in advance, remains the headline comparison; these are extra evidence.

A.3 The Part II embeddings on the Transformer

As an optional extension, the Part I Transformer was also trained with the skip-gram and GloVe tables frozen in (`results/transformer_*_frozen.json`). Its embeddings are multiplied by $\sqrt{200} \approx 14$ before the position signal is added, and it is built for a table at its own initial scale (standard deviation 0.02), so both tables were rescaled by a single constant to that scale, which keeps every vector's direction and the ratios of the distances between them. Everything else matches the Part I Transformer, which leaves 7.7M trainable parameters instead of 13.5M.

Transformer input embeddings	Trainable params	Val PPL	Test PPL	Epochs (best)
Random, trainable (Part I)	13.5M	160.41	149.52	9 (6)
Own skip-gram, frozen	7.7M	138.66	131.17	20 (20)
GloVe, frozen	7.7M	135.10	**127.45**	20 (20)

Table 5: The Part II embedding settings applied to the Transformer.

Both runs used the full 20-epoch budget and were still improving at the end, so their scores are limited by the budget rather than converged. Their validation perplexity at epochs 6, 9, 15 and 20 is 156.78, 151.96, 145.38 and 138.66 with skip-gram, and 160.36, 151.84, 143.96 and 135.10 with GloVe, against the Part I Transformer's best of 160.41 at epoch 6, where it stopped improving. In their last two or three epochs training loss rose slightly while validation loss kept falling; these are the last epochs of the cosine schedule, where the learning rate falls from about 6% of its peak to zero, and the rise is not explained here. Both checkpoints reproduce their recorded losses when reloaded, and both tables are bit-for-bit unchanged after training.

A.4 Representative errors and unknown words (Part III)

The ten most confident errors of each probe include three kinds:

- *Doubtful labels.* Sports stories labelled World are the most common confident error in every probe, for example "Football: Azerbaijan 0-1 England. Michael Owen heads England's winner in the World Cup qualifier" (labelled World, predicted Sports). These are arguably problems of the dataset, not of the model.
- *Genuinely ambiguous topics.* "How Serious Is Pfizer's Condition? The stock is feeling some pain these days" (Business, predicted Sci/Tech) and "Treo 650 on Monday? The blogs are buzzing that the Treo 650 will be released" (Sci/Tech, predicted Business) could each reasonably belong to either class.
- *Model errors.* "Calif. Lawmaker Wants to Privatize State Pensions" (Business) is predicted as Sports with full confidence by the Transformer with last-token pooling.

One possible contributor to the Business and Sci/Tech confusion is unknown words, since both classes are full of company and product names that WikiText-2 does not contain. The data give this only weak support: Sci/Tech has the highest `<unk>` rate (8.1% of tokens, against 6.2% for World), but Business (7.0%) is no

higher than Sports (7.0%); in the best probe, Business items predicted as Sci/Tech contain slightly more `<unk>` than correctly classified ones (7.8% against 7.0%), and Sci/Tech items predicted as Business hardly more (8.4% against 8.3%); in the two LSTM probes, Sci/Tech items predicted as Business contain *fewer* `<unk>` than correctly classified ones.

B Tool use

Claude Opus 5.5 (Anthropic), used through Claude Code, was used to write much of the code, to draft this report from the author's notes, and to verify the results. *Code:* it wrote the configuration loading, data pipeline, RNN/LSTM model, training loop, benchmark, embedding code, AG News preparation, probe, analysis and figure scripts; the author wrote the Transformer and, with review, parts of the Kneser-Ney trigram and of the evaluation, generation and pooling code. The repository README gives the split file by file. *Report:* the assistant produced the tables, figures and measured facts and turned the author's notes into prose. *Reasoning:* the questions asked and the interpretations are the author's; where the assistant suggested an analysis or an explanation (for example the per-position and cold-start analysis, the scale confound and the parameter-count point), the author reviewed it before adopting it.